

Vector Container

Generated by Doxygen 1.17.0

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 Vector< T > Class Template Reference	5
3.1.1 Constructor & Destructor Documentation	7
3.1.1.1 Vector() [1/3]	7
3.1.1.2 ~Vector()	7
3.1.1.3 Vector() [2/3]	7
3.1.1.4 Vector() [3/3]	7
3.1.2 Member Function Documentation	7
3.1.2.1 assign() [1/2]	7
3.1.2.2 assign() [2/2]	8
3.1.2.3 at() [1/2]	8
3.1.2.4 at() [2/2]	8
3.1.2.5 back() [1/2]	9
3.1.2.6 back() [2/2]	9
3.1.2.7 begin() [1/2]	9
3.1.2.8 begin() [2/2]	9
3.1.2.9 capacity()	10
3.1.2.10 clear()	10
3.1.2.11 empty()	10
3.1.2.12 end() [1/2]	10
3.1.2.13 end() [2/2]	10
3.1.2.14 erase() [1/3]	11
3.1.2.15 erase() [2/3]	11
3.1.2.16 erase() [3/3]	11
3.1.2.17 front() [1/2]	12
3.1.2.18 front() [2/2]	12
3.1.2.19 insert()	12
3.1.2.20 operator!=(())	12
3.1.2.21 operator<()	13
3.1.2.22 operator<=()	13
3.1.2.23 operator=() [1/2]	13
3.1.2.24 operator=() [2/2]	13
3.1.2.25 operator==(())	14
3.1.2.26 operator>()	14
3.1.2.27 operator>=()	14
3.1.2.28 operator[]() [1/2]	15
3.1.2.29 operator[]() [2/2]	15

3.1.2.30 pop_back()	15
3.1.2.31 push_back() [1/2]	15
3.1.2.32 push_back() [2/2]	16
3.1.2.33 reserve()	16
3.1.2.34 resize()	16
3.1.2.35 size()	16
3.1.2.36 swap()	17
4 File Documentation	19
4.1 vector.h File Reference	19
4.1.1 Detailed Description	19
4.2 vector.h	19
Index	25

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Vector< T >	5
-----------------------------------	---

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

vector.h	Nuosavo Vector konteinerio realizacija	19
--------------------------	--	----

Chapter 3

Class Documentation

3.1 `Vector< T >` Class Template Reference

```
#include <vector.h>
```

Public Member Functions

- `Vector ()`
Sukuria tuščią konteinerį.
- `~Vector ()`
Destruktorius.
- `Vector (const Vector &other)`
Kopijavimo konstruktorius.
- `Vector (Vector &&other) noexcept`
Move konstruktorius.
- `Vector & operator= (const Vector &other)`
Kopijavimo priskyrimo operatorius.
- `Vector & operator= (Vector &&other) noexcept`
Move priskyrimo operatorius.
- `void push_back (const T &value)`
Prideda elementą į pabaigą.
- `void push_back (T &&value)`
Prideda elementą naudojant move semantiką.
- `size_t size () const`
Grąžina elementų kiekį.
- `T & operator[] (size_t index)`
Pasiekia elementą pagal indeksą.
- `const T & operator[] (size_t index) const`
Pasiekia elementą pagal indeksą.
- `size_t capacity () const`
Grąžina talpą.
- `bool empty () const`
Patikrina ar konteineris tuščias.
- `T & front ()`
Grąžina pirmą elementą.

- T & **back** ()
Grąžina paskutinį elementą.
- const T & **front** () const
Grąžina pirmą elementą const reikšmei.
- const T & **back** () const
Pašalina paskutinį elementą const reikšmei.
- void **pop_back** ()
Pašalina paskutinį elementą.
- void **clear** ()
Išvalo konteinerį.
- void **resize** (size_t newSize)
Pakeičia dydį.
- void **reserve** (size_t newCapacity)
Rezervuoja talpą.
- void **assign** (size_t newSize, const T &value)
Užpildo konteinerį reikšmėmis.
- void **assign** (T *first, T *last)
Priskiria elementus iš intervalo.
- void **insert** (size_t index, const T &value)
Įterpia elementą.
- void **erase** (size_t index)
Pašalina elementą pagal indeksą.
- T * **erase** (T *pos)
Pašalina elementą pagal iteratorių.
- T * **erase** (T *first, T *last)
Pašalina intervalą.
- void **swap** (Vector &other)
Sukeičia konteinerius.
- T * **begin** ()
Iteratorius į pradžią.
- T * **end** ()
Iteratorius į pabaigą.
- T & **at** (size_t index)
Pasiekia elementą su ribų tikrinimu.
- const T & **at** (size_t index) const
Pasiekia elementą su ribų tikrinimu const reikšmei.
- bool **operator==** (const Vector &other) const
Palygina konteinerius.
- bool **operator!=** (const Vector &other) const
Palygina konteinerius.
- bool **operator<** (const Vector &other) const
- bool **operator>** (const Vector &other) const
Patikrina ar dabartinis konteineris didesnis už kitą.
- bool **operator<=** (const Vector &other) const
Patikrina ar dabartinis konteineris mažesnis arba lygus kitam.
- bool **operator>=** (const Vector &other) const
Patikrina ar dabartinis konteineris didesnis arba lygus kitam.
- const T * **begin** () const
Grąžina konstantinį iteratorių į pradžią.
- const T * **end** () const
Grąžina konstantinį iteratorių į pabaigą.

3.1.1 Constructor & Destructor Documentation

3.1.1.1 Vector() [1/3]

```
template<typename T>
Vector< T >::Vector () [inline]
```

Sukuria tuščią konteinerį.

3.1.1.2 ~Vector()

```
template<typename T>
Vector< T >::~~Vector () [inline]
```

Destruktorius.

3.1.1.3 Vector() [2/3]

```
template<typename T>
Vector< T >::Vector (
    const Vector< T > & other) [inline]
```

Kopijavimo konstruktorius.

Parameters

<i>other</i>	kopijuojamas objektas
--------------	-----------------------

3.1.1.4 Vector() [3/3]

```
template<typename T>
Vector< T >::Vector (
    Vector< T > && other) [inline], [noexcept]
```

Move konstruktorius.

Parameters

<i>other</i>	perkeliamas objektas
--------------	----------------------

3.1.2 Member Function Documentation

3.1.2.1 assign() [1/2]

```
template<typename T>
void Vector< T >::assign (
    size_t newSize,
    const T & value) [inline]
```

Užpildo konteinerį reikšmėmis.

Parameters

<i>newSize</i>	dydis
----------------	-------

<i>value</i>	reikšmė
--------------	---------

3.1.2.2 assign() [2/2]

```
template<typename T>
void Vector< T >::assign (
    T * first,
    T * last) [inline]
```

Priskiria elementus iš intervalo.

Parameters

<i>first</i>	pradžia
<i>last</i>	pabaiga

3.1.2.3 at() [1/2]

```
template<typename T>
T & Vector< T >::at (
    size_t index) [inline]
```

Pasiekia elementą su ribų tikrinimu.

Parameters

<i>index</i>	indeksas
--------------	----------

Returns

nuoroda į elementą

3.1.2.4 at() [2/2]

```
template<typename T>
const T & Vector< T >::at (
    size_t index) const [inline]
```

Pasiekia elementą su ribų tikrinimu const reikšmei.

Parameters

<i>index</i>	indeksas
--------------	----------

Returns

nuoroda į elementą

3.1.2.5 back() [1/2]

```
template<typename T>  
T & Vector< T >::back () [inline]
```

Grąžina paskutinį elementą.

Returns

nuoroda į paskutinį elementą

3.1.2.6 back() [2/2]

```
template<typename T>  
const T & Vector< T >::back () const [inline]
```

Pašalina paskutinį elementą const reikšmei.

Returns

nuoroda į paskutinį elementą

3.1.2.7 begin() [1/2]

```
template<typename T>  
T * Vector< T >::begin () [inline]
```

Iteratorius į pradžią.

Returns

iteratorius

3.1.2.8 begin() [2/2]

```
template<typename T>  
const T * Vector< T >::begin () const [inline]
```

Grąžina konstantinį iteratorių į pradžią.

Returns

iteratorius į pirmą elementą

3.1.2.9 capacity()

```
template<typename T>
size_t Vector< T >::capacity () const [inline]
```

Grąžina talpą.

Returns

talpa

3.1.2.10 clear()

```
template<typename T>
void Vector< T >::clear () [inline]
```

Išvalo konteinerį.

3.1.2.11 empty()

```
template<typename T>
bool Vector< T >::empty () const [inline]
```

Patikrina ar konteineris tuščias.

Returns

true jei tuščias

3.1.2.12 end() [1/2]

```
template<typename T>
T * Vector< T >::end () [inline]
```

Iteratorius į pabaigą.

Returns

iteratorius

3.1.2.13 end() [2/2]

```
template<typename T>
const T * Vector< T >::end () const [inline]
```

Grąžina konstantinį iteratorių į pabaigą.

Returns

iteratorius už paskutinio elemento

3.1.2.14 erase() [1/3]

```
template<typename T>
void Vector< T >::erase (
    size_t index) [inline]
```

Pašalina elementą pagal indeksą.

Parameters

<i>index</i>	indeksas
--------------	----------

3.1.2.15 erase() [2/3]

```
template<typename T>
T * Vector< T >::erase (
    T * first,
    T * last) [inline]
```

Pašalina intervalą.

Parameters

<i>first</i>	pradžia
<i>last</i>	pabaiga

Returns

iteratorius po pašalinimo

3.1.2.16 erase() [3/3]

```
template<typename T>
T * Vector< T >::erase (
    T * pos) [inline]
```

Pašalina elementą pagal iteratorių.

Parameters

<i>pos</i>	iteratorius
------------	-------------

Returns

iteratorius į sekantį elementą

3.1.2.17 front() [1/2]

```
template<typename T>
T & Vector< T >::front () [inline]
```

Grąžina pirmą elementą.

Returns

nuoroda į pirmą elementą

3.1.2.18 front() [2/2]

```
template<typename T>
const T & Vector< T >::front () const [inline]
```

Grąžina pirmą elementą const reikšmei.

Returns

nuoroda į pirmą elementą

3.1.2.19 insert()

```
template<typename T>
void Vector< T >::insert (
    size_t index,
    const T & value) [inline]
```

Įterpia elementą.

Parameters

<i>index</i>	pozicija
<i>value</i>	reikšmė

3.1.2.20 operator!=(())

```
template<typename T>
bool Vector< T >::operator!= (
    const Vector< T > & other) const [inline]
```

Palygina konteinerius.

Parameters

<i>other</i>	kitas konteineris
--------------	-------------------

Returns

true jei nevienodi

3.1.2.21 operator<()

```
template<typename T>
bool Vector< T >::operator< (
    const Vector< T > & other) const [inline]
```

3.1.2.22 operator<=()

```
template<typename T>
bool Vector< T >::operator<= (
    const Vector< T > & other) const [inline]
```

Patikrina ar dabartinis konteineris mažesnis arba lygus kitam.

Parameters

<i>other</i>	kitas konteineris
--------------	-------------------

Returns

true jei mažesnis arba lygus

3.1.2.23 operator=() [1/2]

```
template<typename T>
Vector & Vector< T >::operator= (
    const Vector< T > & other) [inline]
```

Kopijavimo priskyrimo operatorius.

Parameters

<i>other</i>	kopijuojamas objektas
--------------	-----------------------

Returns

nuoroda į objektą

3.1.2.24 operator=() [2/2]

```
template<typename T>
Vector & Vector< T >::operator= (
    Vector< T > && other) [inline], [noexcept]
```

Move priskyrimo operatorius.

Parameters

<i>other</i>	perkeliamas objektas
--------------	----------------------

Returns

nuoroda į objektą

3.1.2.25 operator==()

```
template<typename T>
bool Vector< T >::operator== (
    const Vector< T > & other) const [inline]
```

Palygina konteinerius.

Parameters

<i>other</i>	kitas konteineris
--------------	-------------------

Returns

true jei vienodi

3.1.2.26 operator>()

```
template<typename T>
bool Vector< T >::operator> (
    const Vector< T > & other) const [inline]
```

Patikrina ar dabartinis konteineris didesnis už kitą.

Parameters

<i>other</i>	kitas konteineris
--------------	-------------------

Returns

true jei didesnis

3.1.2.27 operator>=()

```
template<typename T>
bool Vector< T >::operator>= (
    const Vector< T > & other) const [inline]
```

Patikrina ar dabartinis konteineris didesnis arba lygus kitam.

Parameters

<i>other</i>	kitas konteineris
--------------	-------------------

Returns

true jei didesnis arba lygus

3.1.2.28 operator[]() [1/2]

```
template<typename T>
T & Vector< T >::operator[] (
    size_t index) [inline]
```

Pasiekia elementą pagal indeksą.

Parameters

<i>index</i>	elemento indeksas
--------------	-------------------

Returns

nuoroda į elementą

3.1.2.29 operator[]() [2/2]

```
template<typename T>
const T & Vector< T >::operator[] (
    size_t index) const [inline]
```

Pasiekia elementą pagal indeksą.

Parameters

<i>index</i>	elemento indeksas
--------------	-------------------

Returns

konstantiška nuoroda

3.1.2.30 pop_back()

```
template<typename T>
void Vector< T >::pop_back () [inline]
```

Pašalina paskutinį elementą.

3.1.2.31 push_back() [1/2]

```
template<typename T>
void Vector< T >::push_back (
    const T & value) [inline]
```

Prideda elementą į pabaigą.

Parameters

<i>value</i>	pridedama reikšmė
--------------	-------------------

3.1.2.32 push_back() [2/2]

```
template<typename T>
void Vector< T >::push_back (
    T && value) [inline]
```

Prideda elementą naudojant move semantiką.

Parameters

<i>value</i>	pridedama reikšmė
--------------	-------------------

3.1.2.33 reserve()

```
template<typename T>
void Vector< T >::reserve (
    size_t newCapacity) [inline]
```

Rezervuoja talpą.

Parameters

<i>newCapacity</i>	nauja talpa
--------------------	-------------

3.1.2.34 resize()

```
template<typename T>
void Vector< T >::resize (
    size_t newSize) [inline]
```

Pakeičia dydį.

Parameters

<i>newSize</i>	naujas dydis
----------------	--------------

3.1.2.35 size()

```
template<typename T>
size_t Vector< T >::size () const [inline]
```

Grąžina elementų kiekį.

Returns

elementų kiekis

3.1.2.36 swap()

```
template<typename T>
void Vector< T >::swap (
    Vector< T > & other) [inline]
```

Sukeičia konteinerius.

Parameters

<i>other</i>	kitas konteineris
--------------	-------------------

The documentation for this class was generated from the following file:

- [vector.h](#)

Chapter 4

File Documentation

4.1 vector.h File Reference

Nuosavo [Vector](#) konteinerio realizacija.

```
#include <iostream>
#include <stdexcept>
#include <utility>
```

Classes

- class [Vector< T >](#)

4.1.1 Detailed Description

Nuosavo [Vector](#) konteinerio realizacija.

4.2 vector.h

[Go to the documentation of this file.](#)

```
00001 #ifndef VECTOR_H
00002 #define VECTOR_H
00003
00004 #include <iostream>
00005 #include <stdexcept>
00006 #include <utility>
00007
00012
00013 template <typename T>
00014 class Vector {
00015 private:
00016     T* data_;
00017     size_t size_;
00018     size_t capacity_;
00019
00020 public:
00021
00025     Vector()
00026         : data_(nullptr), size_(0), capacity_(0) {}
00027
```

```

00031     ~Vector() {
00032         delete[] data_;
00033     } //destruktorius
00034
00035     Vector(const Vector& other)
00036     : data_(nullptr),
00037       size_(other.size_),
00038       capacity_(other.capacity_) {
00039
00040         data_ = capacity_ ? new T[capacity_] : nullptr;
00041
00042         for (size_t i = 0; i < size_; i++) {
00043             data_[i] = other.data_[i];
00044         }
00045     } //copy kons.
00046
00047     Vector(Vector&& other) noexcept
00048     : data_(other.data_), size_(other.size_), capacity_(other.capacity_) {
00049         other.data_ = nullptr;
00050         other.size_ = 0;
00051         other.capacity_ = 0;
00052     } //move kons.
00053
00054     Vector& operator=(const Vector& other) {
00055         if (this != &other) {
00056             delete[] data_;
00057
00058             size_ = other.size_;
00059             capacity_ = other.capacity_;
00060             data_ = capacity_ ? new T[capacity_] : nullptr;
00061
00062             for (size_t i = 0; i < size_; i++) {
00063                 data_[i] = other.data_[i];
00064             }
00065             return *this;
00066         } //copy assigment k.
00067
00068     Vector& operator=(Vector&& other) noexcept {
00069         if (this != &other) {
00070             delete[] data_;
00071
00072             data_ = other.data_;
00073             size_ = other.size_;
00074             capacity_ = other.capacity_;
00075
00076             other.data_ = nullptr;
00077             other.size_ = 0;
00078             other.capacity_ = 0;
00079         }
00080         return *this;
00081     } //move assigment k.
00082
00083     void push_back(const T& value) {
00084         if (size_ == capacity_) {
00085             size_t newCapacity;
00086
00087             if (capacity_ == 0)
00088                 newCapacity = 1;
00089             else
00090                 newCapacity = capacity_ * 2;
00091
00092             T* newData = new T[newCapacity];
00093
00094             for (size_t i = 0; i < size_; i++) {
00095                 newData[i] = data_[i];
00096             }
00097
00098             delete[] data_;
00099
00100             data_ = newData;
00101             capacity_ = newCapacity;
00102         }
00103
00104         data_[size_] = value;
00105         size_++;
00106     } //push_back funkcija
00107
00108     void push_back(T& value) {
00109         if (size_ == capacity_) {
00110             size_t newCapacity;

```



```

00144         if (capacity_ == 0)
00145             newCapacity = 1;
00146         else
00147             newCapacity = capacity_ * 2;
00148
00149         T* newData = new T[newCapacity];
00150
00151         for (size_t i = 0; i < size_; i++) {
00152             newData[i] = std::move(data_[i]);
00153         }
00154
00155         delete[] data_;
00156
00157         data_ = newData;
00158         capacity_ = newCapacity;
00159     }
00160
00161     data_[size_] = std::move(value);
00162     size_++;
00163 } //move push_back funkcija
00164 size_t size() const {
00165     return size_;
00166 } //size getteris
00167 T& operator[](size_t index) {
00168     return data_[index];
00169 } //index operatorius
00170 const T& operator[](size_t index) const {
00171     return data_[index];
00172 } //const index operatorius
00173 size_t capacity() const {
00174     return capacity_;
00175 } //capacity getteris
00176
00177 bool empty() const {
00178     return size_ == 0;
00179 } //empty getteris
00180 T& front() {
00181     if (empty()) {
00182         throw std::out_of_range("Vector is empty");
00183     }
00184     return data_[0];
00185 } //front getteris
00186 T& back() {
00187     if (empty()) {
00188         throw std::out_of_range("Vector is empty");
00189     }
00190     return data_[size_ - 1];
00191 } //back getteris
00192
00193 const T& front() const {
00194     if (empty()) {
00195         throw std::out_of_range("Vector is empty");
00196     }
00197     return data_[0];
00198 } //const front getteris
00199 const T& back() const {
00200     if (empty()) {
00201         throw std::out_of_range("Vector is empty");
00202     }
00203     return data_[size_ - 1];
00204 } //const back getteris
00205
00206 void pop_back() {
00207     if (empty()) {
00208         throw std::out_of_range("Vector is empty");
00209     }
00210     size_--;
00211 } //pop_back funkcija
00212 void clear() {
00213     size_ = 0;
00214 } //clear funkcija
00215 void resize(size_t newSize) {
00216     if (newSize > capacity_) {
00217         size_t newCapacity = capacity_ == 0 ? 1 : capacity_ * 2;
00218         while (newCapacity < newSize) {
00219             newCapacity *= 2;
00220         }
00221
00222         T* newData = new T[newCapacity];
00223
00224         for (size_t i = 0; i < size_; i++) {
00225             newData[i] = data_[i];
00226         }
00227
00228         delete[] data_;
00229
00230         data_ = newData;

```

```

00279         capacity_ = newCapacity;
00280     }
00281     size_ = newSize;
00282 } //resize funckija
00283 void reserve(size_t newCapacity) {
00284     if (newCapacity > capacity_) {
00285         T* newData = new T[newCapacity];
00290
00291         for (size_t i = 0; i < size_; i++) {
00292             newData[i] = data_[i];
00293         }
00294
00295         delete[] data_;
00296
00297         data_ = newData;
00298         capacity_ = newCapacity;
00299     }
00300 } //reserve funckija
00301 void assign(size_t newSize, const T& value) {
00302     if (newSize > capacity_) {
00303         size_t newCapacity = capacity_ == 0 ? 1 : capacity_ * 2;
00304         while (newCapacity < newSize) {
00305             newCapacity *= 2;
00311
00312         }
00313         T* newData = new T[newCapacity];
00314
00315         for (size_t i = 0; i < size_; i++) {
00316             newData[i] = data_[i];
00317         }
00318
00319         delete[] data_;
00320
00321         data_ = newData;
00322         capacity_ = newCapacity;
00323     }
00324     for (size_t i = 0; i < newSize; i++) {
00325         data_[i] = value;
00326     }
00327     size_ = newSize;
00328 } //assign funckija
00329
00330 void assign(T* first, T* last) {
00331     size_t newSize = last - first;
00332
00333     if (newSize > capacity_) {
00334         T* newData = new T[newSize];
00335
00336         delete[] data_;
00337
00338         data_ = newData;
00339         capacity_ = newSize;
00340
00341         for (size_t i = 0; i < newSize; i++) {
00342             data_[i] = first[i];
00343         }
00344
00345         size_ = newSize;
00346
00347     }
00348
00349     void insert(size_t index, const T& value) {
00350         if (index > size_) {
00351             throw std::out_of_range("Index out of range");
00352         }
00353         if (size_ == capacity_) {
00354             size_t newCapacity = capacity_ == 0 ? 1 : capacity_ * 2;
00355             T* newData = new T[newCapacity];
00356
00357             for (size_t i = 0; i < index; i++) {
00358                 newData[i] = data_[i];
00359             }
00360
00361             newData[index] = value;
00362
00363             for (size_t i = index; i < size_; i++) {
00364                 newData[i + 1] = data_[i];
00365             }
00366
00367             delete[] data_;
00368
00369             data_ = newData;
00370             capacity_ = newCapacity;
00371         } else {
00372             for (size_t i = size_; i > index; i--) {
00373                 data_[i] = data_[i - 1];
00374             }
00375
00376             data_[index] = value;
00377         }
00378     }

```

```

00385     }
00386     size_++;
00387 } //insert funckija
00388
00393 void erase(size_t index) {
00394     if (index >= size_) {
00395         throw std::out_of_range("Index out of range");
00396     }
00397     for (size_t i = index; i < size_ - 1; i++) {
00398         data_[i] = data_[i + 1];
00399     }
00400     size_--;
00401 } //erase funckija
00407 T* erase(T* pos) {
00408     size_t index = pos - data_;
00409     erase(index);
00410     return data_ + index;
00411 }
00412
00419 T* erase(T* first, T* last) {
00420     size_t indexFirst = first - data_;
00421     size_t indexLast = last - data_;
00422
00423     if (indexFirst > size_ || indexLast > size_ || indexFirst > indexLast) {
00424         throw std::out_of_range("Invalid erase range");
00425     }
00426
00427     size_t count = indexLast - indexFirst;
00428
00429     for (size_t i = indexFirst; i + count < size_; i++) {
00430         data_[i] = std::move(data_[i + count]);
00431     }
00432
00433     size_ -= count;
00434
00435     return data_ + indexFirst;
00436 }
00441 void swap(Vector& other) {
00442
00443     std::swap(data_, other.data_);
00444     std::swap(size_, other.size_);
00445     std::swap(capacity_, other.capacity_);
00446 } //swap funckija
00447
00452 T* begin() {
00453     return data_;
00454 } //begin getteris
00455
00460 T* end() {
00461     return data_ + size_;
00462 } //end getteris
00468 T& at(size_t index) {
00469     if (index >= size_) {
00470         throw std::out_of_range("Index out of range");
00471     }
00472     return data_[index];
00473 } //at getteris
00479 const T& at(size_t index) const {
00480     if (index >= size_) {
00481         throw std::out_of_range("Index out of range");
00482     }
00483     return data_[index];
00484 } //const at getteris
00485
00491 bool operator==(const Vector& other) const {
00492     if (size_ != other.size_) {
00493         return false;
00494     }
00495     for (size_t i = 0; i < size_; i++) {
00496         if (data_[i] != other.data_[i]) {
00497             return false;
00498         }
00499     }
00500     return true;
00501 } //equality operatorius
00507 bool operator!=(const Vector& other) const {
00508     return !(*this == other);
00509 } //inequality operatorius
00510
00511 bool operator<(const Vector& other) const {
00512     size_t minSize = size_ < other.size_ ? size_ : other.size_;
00513     for (size_t i = 0; i < minSize; i++) {
00514         if (data_[i] < other.data_[i]) {
00515             return true;
00516         } else if (data_[i] > other.data_[i]) {
00517             return false;
00518         }

```

```
00519     }
00520     return size_ < other.size_;
00521 } //less than operatorius
00527 bool operator>(const Vector& other) const {
00528     return other < *this;
00529 } //greater than operatorius
00535 bool operator<=(const Vector& other) const {
00536     return !(*this > other);
00537 } //less than or equal operatorius
00543 bool operator>=(const Vector& other) const {
00544     return !(*this < other);
00545 } //greater than or equal operatorius
00550 const T* begin() const {
00551     return data_;
00552 } //const begin getteris
00557 const T* end() const {
00558     return data_ + size_;
00559 } //const end getteris
00560 };
00561 #endif
```

Index

`~Vector`
 `Vector< T >`, 7

`assign`
 `Vector< T >`, 7, 8

`at`
 `Vector< T >`, 8

`back`
 `Vector< T >`, 8, 9

`begin`
 `Vector< T >`, 9

`capacity`
 `Vector< T >`, 9

`clear`
 `Vector< T >`, 10

`empty`
 `Vector< T >`, 10

`end`
 `Vector< T >`, 10

`erase`
 `Vector< T >`, 10, 11

`front`
 `Vector< T >`, 11, 12

`insert`
 `Vector< T >`, 12

`operator!=`
 `Vector< T >`, 12

`operator<`
 `Vector< T >`, 12

`operator<=`
 `Vector< T >`, 13

`operator>`
 `Vector< T >`, 14

`operator>=`
 `Vector< T >`, 14

`operator=`
 `Vector< T >`, 13

`operator==`
 `Vector< T >`, 13

`operator[]`
 `Vector< T >`, 14, 15

`pop_back`
 `Vector< T >`, 15

`push_back`
 `Vector< T >`, 15

`reserve`
 `Vector< T >`, 16

`resize`
 `Vector< T >`, 16

`size`
 `Vector< T >`, 16

`swap`
 `Vector< T >`, 16

`Vector`
 `Vector< T >`, 7

`Vector< T >`, 5

`~Vector`, 7

`assign`, 7, 8

`at`, 8

`back`, 8, 9

`begin`, 9

`capacity`, 9

`clear`, 10

`empty`, 10

`end`, 10

`erase`, 10, 11

`front`, 11, 12

`insert`, 12

`operator!=`, 12

`operator<`, 12

`operator<=`, 13

`operator>`, 14

`operator>=`, 14

`operator=`, 13

`operator==`, 13

`operator[]`, 14, 15

`pop_back`, 15

`push_back`, 15

`reserve`, 16

`resize`, 16

`size`, 16

`swap`, 16

`Vector`, 7

`vector.h`, 19